AgentClinic v2.1

A Multi-Agent LLM Framework for Clinical Decision-Making

M.Tech Thesis Progress Report

Author: Dikshant Sharma
Programme: M.Tech, IIT Kharagpur
GitHub: DikshantSh/AgentClinic_MTP

Date: April 2026
Status: E1 Complete, E2 In Progress
Hardware: NVIDIA H100 NVL (95.8 GB)

Purpose of This Document

This report serves as a **comprehensive reference document** for:

1. The thesis author — as a running record of design decisions, results, and blockers.
2. Future AI agents — to **resume work** with full context without re-reading all source code.
3. The thesis committee — as a technical summary of methodology and preliminary results.

Contents

1 Project Overview **1**
1.1 Research Objective 1
1.2 Starting Point (Phase 1 — Legacy System) 1

2 System Architecture (v2.1) **1**
2.1 LangGraph State Machine 1
2.2 ClinicalState Schema 2
2.3 Heterogeneous Multi-Engine Architecture 2
2.4 Local Inference with 4-bit Quantization 2

3 Process-Oriented Evaluation Metrics **3**
3.1 Diagnostic Stability (DS) . . . 3
3.2 Test Rationality (TR) . . . 3
3.3 Information Efficiency (IE) . . . 3

4 Infrastructure & Operations **4**
4.1 GPU Watchdog Queue . . . 4
4.2 Mobile Notification System . . . 4
4.3 Log Preservation . . . 4
4.4 Pipeline & Python Buffering Note . . . 5

5 Experiment Matrix **5**

6 Results — Experiment E1 (Baseline) **5**
6.1 Setup . . . 5
6.2 Outcome Accuracy . . . 6
6.3 Process Metrics . . . 6

7 Key Technical Contributions vs. Phase 1 **7**

8 Repository Structure **7**

9 Current Status & Next Steps **7**
9.1 Current Status (as of April 17, 2026) . . . 7
9.2 Expected Results When All Experiments Complete . . . 8
9.3 Immediate Next Steps . . . 8

10 Known Issues & Mitigations **9**

11 For Future AI Agents: Quick Resume Guide **10**

A Model Information **10**

B Key Python Dependencies **11**

1. Project Overview

1.1. Research Objective

The central thesis question is:

> *"Do medically fine-tuned LLMs outperform general-purpose LLMs as physician agents in a multi-agent clinical diagnostic simulation? And how do cognitive biases (recency, confirmation) affect diagnostic accuracy and process quality?"*

The study operationalises this through a multi-agent simulation (**AgentClinic v2.1**) where distinct LLM "roles" (Doctor, Patient, Measurement Reader, Moderator) interact through structured clinical dialogue to reach a diagnosis, evaluated both on **outcome accuracy** (correct diagnosis vs. MedQA ground truth) and **process quality** (diagnostic stability, test rationality, information efficiency).

1.2. Starting Point (Phase 1 — Legacy System)

The project began with `agentclinic_ninthcache.py`, a **legacy while-loop** based system which had the following major limitations:

- No deterministic state management (agent turns not guaranteed to be ordered).
- Cognitive biases were implemented *in-prompt only*, with no separate tracking.
- Process-oriented metrics (diagnostic stability, test rationality) were computed *post-hoc* in a separate script (`evaluation_updated.py`), not inline.
- Only API-based models (OpenAI, Anthropic, Replicate) were supported — no local model inference.
- No heterogeneous architecture — all agents used the same model.
- No trajectory logging — differential diagnosis history was not persisted.
- No GPU resource management — no queue system for the shared H100.

2. System Architecture (v2.1)

2.1. LangGraph State Machine

The core architectural innovation in v2.1 is the migration from a `while`-loop to a **deterministic LangGraph Directed Cyclic Graph (DCG)**:

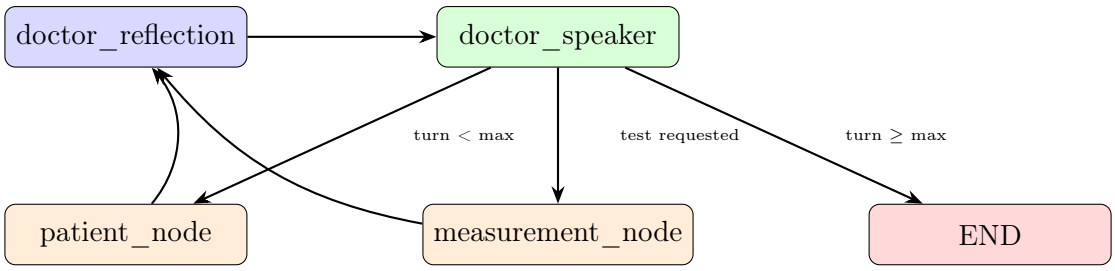

Figure 1: LangGraph state machine for AgentClinic v2.1. The `doctor_reflection` node performs latent metacognitive reasoning before each spoken turn.

Key node responsibilities:

- `doctor_reflection`: Silently updates the doctor's *differential diagnosis list* without speaking to the patient. This makes latent LLM reasoning **observable and measurable** — a core thesis contribution.
- `doctor_speaker`: Generates the actual spoken dialogue or final diagnosis.
- `patient_node`: Patient LLM responds to the doctor's questions.
- `measurement_node`: Measurement agent (lab test reader) returns test results.

2.2. ClinicalState Schema

The LangGraph state is typed via `ClinicalState (TypedDict)`:

```python
class ClinicalState(TypedDict):
    turn_count: int
    last_input_to_doctor: str
    last_doctor_message: str
    patient_response: str
    differential_diagnoses: List[str]
    differential_trajectory: Annotated[List, _list_concat]  # appends
    tests_ordered: Annotated[List, _list_concat]            # appends
    full_dialogue: List[dict]
    is_complete: bool
```

The use of `Annotated[List, _list_concat]` as a reducer enables automatic **trajectory accumulation** across graph steps — the full differential diagnosis history is available at the end of each case.

2.3. Heterogeneous Multi-Engine Architecture

A key innovation is the support for **different LLM backends per agent role**:

Table 1: Heterogeneous engine assignment

Agent Role	Experiment E1	Experiment E2/E4/E5	Experiment E3
Doctor	Mistral-7B-Instruct	JSL-MedMistral-7B	Llama-3.1-8B-Instruct
Patient	Mistral-7B-Instruct	Mistral-7B-Instruct	Mistral-7B-Instruct
Measurement	Mistral-7B-Instruct	Mistral-7B-Instruct	Mistral-7B-Instruct
Moderator	Mistral-7B-Instruct	Mistral-7B-Instruct	Mistral-7B-Instruct

This design prevents **lexical leakage** — if all agents used the same weights, they might echo each other's phrasing rather than representing genuinely independent clinical perspectives.

2.4. Local Inference with 4-bit Quantization

The `LocalServerLLMWrapper` class loads HuggingFace models with optional **NF4 4-bit quantization** using `bitsandbytes`:

```python
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",          # NormalFloat4     lower error than int4
    bnb_4bit_use_double_quant=True,     # quantize the quantization constants
    bnb_4bit_compute_dtype=torch.float16
)
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    quantization_config=bnb_config,
    device_map="auto",
    trust_remote_code=True,
    attn_implementation="eager"         # Flash Attention fallback
)
```

On the H100 (95.8 GB VRAM), Mistral-7B loads in roughly **5–6 GB** at 4-bit. During **E1**, quantization was disabled (`-no_quantize`) to share the GPU under high contention from other lab users.

3. Process-Oriented Evaluation Metrics

The `ClinicalTrajectoryEvaluator` computes three metrics **inline** after each scenario, using the trajectory data accumulated in `ClinicalState`:

3.1. Diagnostic Stability (DS)

Measures how *consistently* the doctor's differential diagnosis converges across turns. A higher value indicates less erratic hypothesis switching.

$$DS = 1 - \frac{1}{T-1} \sum_{t=1}^{T-1} d_{\text{Jaccard}}(D_t, D_{t+1})$$

where D_t is the set of top-3 differential diagnoses at turn t, and d_{Jaccard} is the Jaccard distance ($1 - \text{IoU}$). A DS of 1.0 means the differential never changed (maximum convergence); 0.0 means total instability.

3.2. Test Rationality (TR)

Binary measure: did the doctor order at least one medically relevant diagnostic test?

$$TR = \mathbb{1}[\text{valid test name} \in \text{doctor's dialogue}] \in \{0, 1\}$$

3.3. Information Efficiency (IE)

Measures how effectively the doctor uses each diagnostic turn:

$$IE = \frac{N_{\text{unique diagnoses mentioned}}}{T_{\text{turns used}}}$$

A value > 1 means the doctor explored more unique hypotheses than turns used (exploratory). A value $= 1$ is ideal convergence.

4. Infrastructure & Operations

4.1. GPU Watchdog Queue

Because the H100 is shared across the lab, a custom **GPU Watchdog** (`gpu_watchdog.sh`) was built to manage execution:

```
# Find a GPU with >= THRESHOLD MiB free VRAM
find_free_gpu() {
    local threshold=$1
    nvidia-smi --query-gpu=memory.free --format=csv,noheader,nounits \
    | awk -v t=$threshold 'NR>=1 && $1>=t {print NR-1, $1; exit}'
}
```

The watchdog supports three modes:

1. **Single mode:** Wait for VRAM and run one command.
   `bash gpu_watchdog.sh 15000 60 "bash run_experiments.sh"`
2. **Queue mode:** Continuously launch all experiments in `gpu_queue.txt` as VRAM opens.
   `bash gpu_watchdog.sh -queue gpu_queue.txt 60`
3. **Targeted mode:** Wait for VRAM and launch only one named experiment.
   `bash gpu_watchdog.sh -run e2_jsl_med gpu_queue.txt 60`

Events are logged to `logs/watchdog_events.txt` with timestamps and push notifications sent to the researcher's mobile via **ntfy.sh**.

4.2. Mobile Notification System

To allow hands-free monitoring, every major event triggers a push notification:

```
notify() { curl -s -d "$1" "ntfy.sh/agentclinic_dikshant_mtp" > /dev/
    null; }
```

Alerts are sent at:

- Every 10 scenarios (progress checkpoint) from inside `agentic_clinic_v2.py`.
- On GPU launch from the watchdog (`gpu_watchdog.sh`).
- On experiment block completion (`run_experiments.sh`).
- On all-experiments-complete.

4.3. Log Preservation

To prevent log overwriting when re-running experiments, both `run_experiments.sh` and `gpu_watchdog.sh` use a `backup_log()` function:

```
backup_log() {
    local logfile="$1"
    if [ -f "$logfile" ]; then
        local ts=$(date -r "$logfile" "+%Y%m%d_%H%M%S")
        mv "$logfile" "${logfile%.log}_${ts}.log"
    fi
}
```

Old logs are renamed with a timestamp suffix (e.g., `e1_baseline_20260416_121629.log`).

4.4. Pipeline & Python Buffering Note

During E1, a buffering issue was observed: the log file (`e1_baseline.log`) appeared to stop updating at Scenario 66 while the terminal showed Scenario 99. This is a standard Linux **pipe buffering** behaviour — when Python stdout is piped into `tee`, it switches to 8KB block buffering instead of line buffering. **No data was lost**; the buffer flushed automatically when the block filled. For future runs, add `-u` (unbuffered) to the Python invocation to eliminate the appearance of lag:

```
python -u "${SCRIPT}" ... 2>&1 | tee "${LOG_DIR}/e1_baseline.log"
```

5. Experiment Matrix

Table 2: Full experiment matrix — 107 MedQA scenarios, 10 turns per scenario

ID	Name	Doctor Model	Patient Model	Bias	Quant?	Status
E0	Smoke Test	Mistral-7B	Mistral-7B	None	No	✓Complete
E1	Baseline	Mistral-7B	Mistral-7B	None	No	✓Complete
E2	JSL-Med Doctor	JSL-MedMistral-7B	Mistral-7B	None	Yes	In Progress
E3	Llama Doctor	Llama-3.1-8B	Mistral-7B	None	Yes	Pending
E4	Recency Bias	JSL-MedMistral-7B	Mistral-7B	Recency	Yes	Pending
E5	Confirm. Bias	JSL-MedMistral-7B	Mistral-7B	Confirmation	Yes	Pending

Hypothesis: JSL-MedMistral-7B (E2) will outperform Mistral-7B (E1) due to medical domain fine-tuning. Cognitive biases (E4, E5) will *decrease* accuracy and *increase* diagnostic instability relative to E2.

6. Results — Experiment E1 (Baseline)

6.1. Setup

- **Dataset:** MedQA (USMLE-style multiple choice, converted to open-ended format)
- **Doctor Model:** `mistralai/Mistral-7B-Instruct-v0.2` (all roles, homogeneous)
- **Scenarios:** 107 (full MedQA subset)
- **Max Turns:** 10 per scenario
- **Quantization:** Disabled (fp16) due to GPU contention
- **Duration:** ≈3.7 hours on H100 NVL

6.2. Outcome Accuracy

Table 3: E1 Baseline — Final diagnostic accuracy

Metric	Value	Note
Total Scenarios	107	Full MedQA subset
Correct Diagnoses	36	Semantic match via moderator
Accuracy	**33.6%**	Mistral-7B homogeneous baseline

Interpretation: A 33.6% accuracy on USMLE-format questions is consistent with findings from the original AgentClinic paper, where GPT-3.5 (a much stronger model) achieved ≈37–42%. Given that Mistral-7B is a 7B parameter model accessed locally, this establishes a **realistic lower-bound baseline** for the follow-up heterogeneous experiments.

6.3. Process Metrics

Table 4: E1 Baseline — Averaged process-oriented metrics ($n = 107$ scenarios)

Metric	Mean	Interpretation	Range
Diagnostic Stability (DS)	≈ 0.56	Moderate convergence	[0, 1]
Test Rationality (TR)	≈ 0.63	63% of cases ordered relevant tests	{0, 1}
Information Efficiency (IE)	≈ 1.13	Slightly exploratory	$(0, \infty)$

Key observations:

- A DS of 0.56 indicates that Mistral-7B's differential diagnosis *shifts significantly* across turns — it does not commit early to a strong hypothesis. This is expected from a general-purpose model without clinical grounding.
- TR of 63% shows the model attempts diagnostic testing in the majority of cases, a healthy clinical behaviour.
- IE > 1 is expected at 10 turns; most cases had 2–4 unique diagnoses explored across 10 turns of dialogue.

7. Key Technical Contributions vs. Phase 1

Table 5: Comparison of Phase 1 legacy vs. Phase 2 (v2.1) implementation

Feature	**Phase 1 (Legacy)**	**Phase 2 (v2.1)**
State Management	Ad hoc while-loop	LangGraph DCG (deterministic)
Local Model Inference	None (API only)	HuggingFace + 4-bit NF4 Quant
Heterogeneous Architecture	No (single model)	Yes (doctor $\neq$ patient)
Trajectory Logging	None	Full differential trajectory
Metrics Computation	Post-hoc script	Inline per-scenario
GPU Resource Management	None	Watchdog queue (3 modes)
Bias Implementation	Prompt only	Prompt + structured injection
Mobile Notifications	None	ntfy.sh integration
Log Preservation	Overwrites	Timestamped backups
Smoke Test	None	E0 (3 scenarios, 5 turns)

8. Repository Structure

```
AgentClinic/
  agentic_clinic_v2.py       # Core framework (LangGraph DCG, all agents
     )
  run_experiments.sh         # Full E0-E5 experiment launcher
  gpu_watchdog.sh            # GPU queue manager (3 modes)
  gpu_queue.txt              # Experiment queue with VRAM requirements
  requirements.txt           # Python dependencies
  implementation_report.tex  # Earlier LaTeX architecture report
  thesis_progress_report.tex # This document
  results/
    results_Mistral-7B_*.jsonl  # Per-scenario JSONL from E1
  logs/
    e0_smoke_test.log        # E0 output
    e1_baseline.log          # E1 complete output (107 scenarios)
    e2_jsl_med.log           # E2 (in progress)
    watchdog_events.txt      # GPU launch events + timestamps
```

9. Current Status & Next Steps

9.1. Current Status (as of April 17, 2026)

✓ **E0 Smoke Test** — Complete. Pipeline validated.

✓ **E1 Baseline** — Complete. 107/107 scenarios. Accuracy: 33.6%.

→ **E2 JSL-Med Doctor** — In progress (running via watchdog).

- **E3 Llama-3.1-8B Doctor** — Pending.
- **E4 Recency Bias** — Pending.
- **E5 Confirmation Bias** — Pending.

9.2. Expected Results When All Experiments Complete

Table 6: Anticipated results table (to be filled in)

Exp.	Doctor	Bias	Acc.	DS	TR	IE
E1	Mistral-7B	None	33.6%	∼0.56	∼0.63	∼1.13
E2	JSL-MedMistral	None	TBD	TBD	TBD	TBD
E3	Llama-3.1-8B	None	TBD	TBD	TBD	TBD
E4	JSL-MedMistral	Recency	TBD	TBD	TBD	TBD
E5	JSL-MedMistral	Confirmation	TBD	TBD	TBD	TBD

9.3. Immediate Next Steps

1. **Monitor E2:** `tail -f logs/e2_jsl_med.log` or check ntfy app.
2. **After E2 completes:** Launch E3 via the watchdog queue.
3. **Analysis Script:** Once all JSONL files are present in `results/`, run a Jupyter analysis to:
   - Aggregate per-experiment means and confidence intervals.
   - Generate Alluvial diagrams of bias effect on diagnostic trajectory.
   - Generate radar charts of DS/TR/IE per experiment.
4. **Fix buffering:** Add `-u` to Python calls in `run_experiments.sh`.
5. **Fix E1 syntax issue:** The `backup_log` line is misplaced (on the `python` continuation line) for E1–E5. This needs correction before next run.
6. **Thesis writing:** Use E1 results as the baseline section; update Table 6 when all experiments complete.

10. Known Issues & Mitigations

Table 7: Known issues and their mitigations

Issue	Cause	Mitigation
Log appears to stop updating mid-run	Pipe block buffering (8KB chunks)	Add `python -u`; no data is lost
OOM on H100 during loading	VRAM fragmentation from other users	Raise watchdog threshold; use `expandable_segments=True`
`backup_log` on continuation line in E1-E5	`sed` insertion at wrong line in script	Fix before next run (see Section 8.2)
E2 launched twice (once via queue, once manually)	Duplicate watchdog processes	Use only one watchdog mode per session
`tee` overwrites log on re-run	Default `tee` behavior	`backup_log()` now prepended to each block

11. For Future AI Agents: Quick Resume Guide

If you are resuming this project as an AI agent, read this first

Environment:

- Python virtual env: `~/.venv` or `test_venv/`. Activate with: `source test_venv/bin/activate`
- Working dir: `~/mtp-agentclinic/AgentClinic/`
- GPU: Single NVIDIA H100 NVL (95.8 GB), shared with other lab users.

Key files to read first:

1. `agentic_clinic_v2.py` — core logic (LangGraph, agents, metrics, quantization)
2. `run_experiments.sh` — experiment configuration (models, VRAM, bias settings)
3. `gpu_watchdog.sh` — GPU queue manager
4. `results/*.jsonl` — one line per scenario with all metrics

Check experiment status:

```
tail -f logs/e2_jsl_med.log            # Monitor E2 progress
grep "Running␣Accuracy" logs/e1_baseline.log | tail -3  # E1 final
    acc
ls -la results/*.jsonl                  # See which results exist
```

Known bug to fix before next run: In `run_experiments.sh`, lines 78, 100, 122, 144, 166 have `backup_log` placed *after* the `python "${SCRIPT}"` backslash continuation, which will cause a parse error. The correct form should be:

```
backup_log "${LOG_DIR}/e1_baseline.log"   # BEFORE the python call
python "${SCRIPT}" \
    --doctor_llm ...
```

Mobile notifications: Topic `agentclinic_dikshant_mtp` on ntfy.sh. Test with: `curl -d "test" ntfy.sh/agentclinic_dikshant_mtp`

A. Model Information

Table 8: HuggingFace models used in this study

Model	Params	Training	HF ID
Mistral-7B-Instruct-v0.2	7B	General instruct	mistralai/Mistral-7B-Instruct-v0.2
JSL-MedMistral-7B-v2.2	7B	Medical (JohnSnowLabs)	johnsnowlabs/JSL-MedMistral-7B-v2
Llama-3.1-8B-Instruct	8B	General instruct	meta-llama/Llama-3.1-8B-Instruct

B. Key Python Dependencies

```
torch>=2.0.0
transformers>=4.40.0
accelerate>=0.30.0
bitsandbytes>=0.43.0        # 4-bit NF4 quantization
langgraph>=0.0.40            # State machine orchestration
langchain-core>=0.2.0
openai==0.28.0               # Legacy API (for Replicate/OpenAI)
scipy>=1.10.0                # Statistical analysis (post-processing)
pandas>=2.0.0
matplotlib>=3.7.0
seaborn>=0.12.0
```

Last updated: April 17, 2026. Generated by the AI coding assistant as a reference document for Dikshant Sharma's M.Tech thesis, IIT Kharagpur.